

Improving Step Accuracy in Specification-Driven Development



Classical engineering applied to LLM builds — decomposition, hard tests, and an optimized build graph

Ed Barlow Web Cloud Studio July 2026

Improving Step Accuracy in Specification-Driven Development

Ed Barlow — Web Cloud Studio

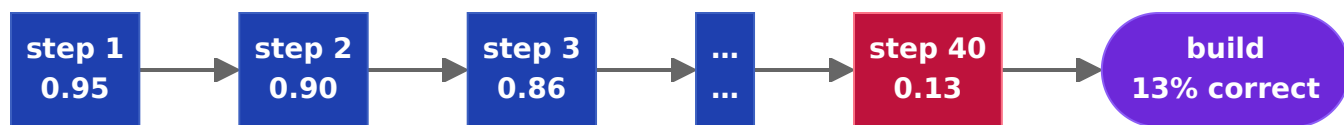
Abstract

Specification-driven development (SDD) builds software from a written specification by executing LLM build steps. Two failure modes dominate: error compounds across unverified steps, and model quality degrades as context grows. This paper applies classical engineering to both. Break the specification into stories by Agile decomposition. Put executable tests on each story. Relate the stories in a graph database and let test results gate the graph. Stack each prompt from delimited specification blocks, and compress a specification to its contract after its first use. Every step is bounded, every step is verified, and the build repeats: the same specification produces the same working software.

Keywords: specification-driven development, LLM code generation, test-driven development, Agile decomposition, graph database, prompt stacking, context compression

1. The Accuracy Problem

A build is a chain of dependent steps. If each step is correct with probability p , the chain is correct with probability p^n . At $p = 0.95$, forty steps deliver a correct build less than 13% of the time.



Unverified error compounds: each step multiplies the survival probability of the build.

The arithmetic applies to every multi-step form:

- An agent chaining n tool calls.
- A pipeline of n build prompts.
- One giant prompt. The model still generates the application serially — file after file, decision after decision. The steps are internal and invisible, but they are steps, and each one can be wrong. Nothing about a single prompt removes the exponent; it only removes the ability to verify between steps.

The second failure mode is context growth. A 250,000-token specification fits inside a modern context window, but fitting is not comprehension. As context grows, measured model behavior degrades [4]:

- Constraints stated early are missed.
- Similar sections are conflated.
- Material is weighted by position, not relevance.

A step prompted with the full specification starts with a lower p than the same step prompted with only what it needs — before any compounding begins.

Both failure modes yield to the same engineering move: break the problem into smaller chunks, put hard tests on each chunk, and surface missing information as questions instead of guesses.

2. Simplification #1: Agile Epic Decomposition

The standard way to break an Epic into stories is Agile. The LLM understands Agile because it is well documented. The specification is the Epic. Epic → features → stories. Decompose until each story builds in one bounded step.

Every story specification ends with four sections:

Section	Contents
Programmatic Acceptance	Executable assertions that define done (§3)
User Acceptance	Checks only a human can honestly judge
Guardrails	What the software must never do
Open Questions	Missing information, returned to the product owner

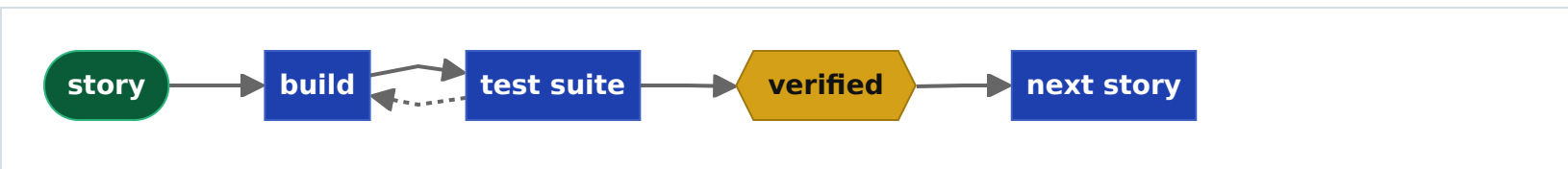
A story that cannot be given Programmatic Acceptance is not yet a story; decompose further.

Open Questions is the feedback channel. Decomposition exposes what the specification does not say; those gaps are written as questions and answered by a human before the story builds. The mechanism for collecting answers varies and is not specified here. The requirement is that ambiguity is resolved by a person, never guessed by the model.

3. Simplification #2: Test-Driven Development for Story Quality

Test-driven development supplies the per-story discipline: write the test before the code; the test suite — not the author — decides when the work is done.

- Programmatic Acceptance is written at decomposition time, as Pythonic, executable assertions: concrete checks against files, routes, return values, and observable behavior.
- Prose criteria ("the import should work correctly") are not acceptance. An assertion that cannot execute cannot gate a build.
- The assertions enter the build prompt as the step's success condition. The task changes from interpreting an instruction to satisfying declared assertions.
- After the step, the test suite runs outside the model. The model is never asked whether it finished; the suite passes or the story fails.



Each story builds, then its test suite executes. Failure loops back; a verified story unlocks the next.

4. Simplification #3: Relate Features and Stories in a Graph Database

Stories are not a list; they are a graph. Features and stories are nodes; dependencies are edges; the graph is stored as plain text alongside the specification. The graph does three jobs:

1. **Ordering.** The runnable frontier — stories whose dependencies have all passed — is computable by inspection. Build order is a property of the data.
2. **Gating.** A story runs only when its dependencies have verified. A failed story blocks its dependents; the defect is caught where it was created and repaired locally.
3. **Containment.** With a test suite at every edge, no unverified chain exceeds length one. The §1 arithmetic collapses from p^n to p per step.

5. Stacking Specifications: Stack and Branding Rules

A build prompt is assembled, not written. Each step stacks the exact files it requires, each wrapped in an XML delimiter naming the file and its role:

```
<pblock filename="FEATURE-Import.md" role="implements">
  ...specification content...
</pblock>
<pblock filename="python.md" role="stack">
  ...stack rules...
</pblock>
```

- The delimiters give the model an unambiguous map of what each block is and why it is present.
- Prompt composition is deterministic: the prompt for any step is a computed function of the graph, reproducible byte for byte.
- Shared stack rules (language, framework, platform conventions) and branding (palette, typography, document standards) are written once and stacked into every step that needs them. No step receives rules irrelevant to its technology.

6. Compression: Second Use Is the Contract

The first story that implements a specification file needs all of it. Every later story needs only the contract:

Full specification	Compressed contract
Schemas, migrations, rationale, examples, internal design	Routes, class names, method signatures, typed parameters, one-line summaries

Each specification file gains a compact derivative containing only its callable surface. The builder step stacks the full file, once; every consumer step stacks the derivative. Compression attacks both §1 failure modes: context per step falls, and what remains is exactly what the step consumes.

7. Optimization: Repeatable Quality Builds

The pieces compose:

Piece	Contribution
Decomposition (§2)	Every step is small enough to be accurate
Test suites (§3)	Every step proves itself before anything depends on it
Graph (§4)	Order is computed; errors are contained at edges
Stacking (§5)	Every prompt is a deterministic function of declared files
Compression (§6)	Every prompt carries contracts, not bulk

Two optimizations fall out of the graph directly:

- Stories that share context group into a single step; the shared material is injected once.
- A change to one specification file invalidates only the stories that depend on it. The rebuild is the affected subgraph, not the application.

Repeatability is the sum. The specification, the graph, the test suites, and the stacking rules fully determine every prompt and every acceptance decision. Run the build again and the same inputs produce the same verified software.

Drydock [1] is the reference implementation of this method.

8. Conclusion

This is classical engineering applied to a new build tool:

- Break the specification into stories small enough to be accurate.
- Put an executable test suite on every story.
- Relate the stories in a graph and let the test suites gate it.
- Stack every prompt from delimited, versioned blocks; compress what is merely consumed.
- Surface what is missing as questions for a human.

Error stops compounding, context stops confusing, and the build repeats.

References

- [1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026. <https://github.com/webcloudstudio/Drydock>
- [2] K. Beck et al. *Manifesto for Agile Software Development*. 2001. <https://agilemanifesto.org>
- [3] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [4] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.